Performance Optimization Report

ExamForge AI | Generated: 2026-08-02

Executive Summary

The performance audit identified 27 findings across 12 categories, including 3 Critical, 8 High, 10 Medium, and 6 Low severity issues. The most impactful findings are: the entire app layout is a client component (preventing RSC benefits for all authenticated pages), the analytics and notifications pages use client-side data fetching instead of server-side, and recharts (~200KB) is statically imported instead of dynamically imported. Additionally, TanStack Query is configured but never used by any client component, meaning all client-side data fetching misses caching, deduplication, and background refetching benefits.

Category	Critical	High	Medium	Low
React Rendering	0	0	3	2
RSC Boundaries	1	0	0	0
Client Components	2	1	0	1
Suspense Boundaries	0	2	0	0
Dynamic Imports	0	2	2	0
Bundle Size	0	0	3	1
Lazy Loading	0	0	1	1
Image Optimization	0	0	1	2
Query Caching	0	1	0	1
Memoization	0	0	2	1

Critical Findings

PF-2.1: Entire App Layout Is a Client Component

The "use client" directive on src/app/(app)/layout.tsx means the entire authenticated layout tree is a client component. This forces all children to be client components by default, losing RSC benefits. The RealtimeProvider could be a wrapper at a lower level instead. This is the single most impactful performance fix because it would unlock server-side rendering for all authenticated pages, reducing Time to First Byte and improving SEO.

Fix: Split into a Server Component layout that wraps a Client Component shell (AppShell).

PF-3.1: Analytics Page Uses Client-Side Data Fetching

The entire analytics page is a "use client" component that fetches data via useEffect + fetch(). This should be a Server Component that fetches data on the server, with only the interactive parts (date range selector, tabs) as client components. Users currently see a loading spinner before any content appears, missing SSR benefits entirely.

PF-3.2: Notifications Page Uses Client-Side Data Fetching

Same pattern as the analytics page. The realtime subscription is a valid reason for client-side code, but the initial data fetch should be server-side. The page should split into a Server Component for initial data and a Client Component for realtime updates.

High Severity Findings

PF-5.1: Recharts Not Dynamically Imported

Recharts (~200KB gzipped) is imported statically in both chart components. These chart components are only used on the /analytics page, meaning every user pays the cost of downloading recharts even if they never visit analytics. Use next/dynamic with ssr: false for chart components.

PF-5.2: Framer Motion in Frequently-Used Component

framer-motion (~30KB gzipped) is imported in StatCard, which is used on nearly every page. This means framer-motion is included in the main bundle. The animations are simple enough to replace with CSS transitions (hover: -translate-y-0.5 transition-all duration-300).

PF-4.1: Missing Per-Route loading.tsx Files

Only the root (app)/loading.tsx exists. Individual routes like /analytics, /cbt, /results, /notifications, /settings, etc. have no loading.tsx. This means route transitions use the generic app shell loading skeleton instead of a route-specific skeleton, causing layout shift.

PF-4.2: No Suspense Boundaries for Streaming

No Suspense boundaries are used anywhere in the app. For pages with multiple async data fetches, wrapping sections in Suspense would allow progressive streaming: the header and stats can appear while activities are still loading.

PF-11.1: TanStack Query Configured But Never Used

The QueryProvider is set up with sensible defaults (60s staleTime, 5min gcTime, structural sharing), but no component uses useQuery, useMutation, or useQueryClient. All client-side data fetching uses raw useEffect + fetch/supabase, missing caching, deduplication, and background refetching.

PF-3.3: 701-Line Settings Client Component

The settings page is a 701-line client component. While it needs interactivity (forms), the static parts (tab labels, theme selector UI) could be server-rendered. The page should be split into a Server Component for initial data and 4 separate client components for each tab.

Recommended Action Plan

Priority	Action	Estimated Impact
P0	Split (app)/layout.tsx into Server + Client shell	Unlocks RSC for all authenticated pages
P0	Refactor Analytics to Server Component	SSR, faster FCP, no loading spinner
P0	Refactor Notifications to Server + Client	SSR for initial data, client for realtime

Priority	Action	Estimated Impact
P1	Dynamic import recharts	~200KB saved from main bundle
P1	Remove unused packages	~700KB+ saved from bundle
P1	Add per-route loading.tsx	Reduced layout shift, better perceived performance
P1	Replace framer-motion in StatCard	~30KB saved from main bundle
P1	Use TanStack Query for client-side fetching	Caching, deduplication, refetching
P2	Add Suspense boundaries	Progressive streaming, better UX
P2	Add React.memo to nav items	Reduced re-renders on sidebar toggle